

SafeGuard: A Real-time Weapon Detection System.

Pedro Monteiro^{1,4} pmapm@ua.pt, Osvaldo Rocha Pacheco^{1,2,4} orp@ua.pt, Gonalo Carnaz^{2,3,4} goncalo.carnaz@ua.pt

1. DETI - Departamento de Electr nica, Telecomunica  es e Inform tica; 2. IEETA - Instituto de Engenharia Electr nica e Telem tica de Aveiro; 3. ISCA - Instituto Superior de Contabilidade e Administra  o; 4. UA - Universidade de Aveiro

Introduction

An increase in firearm-related crimes has been observed in recent years, as evidenced by the RASI report [1]. This trend is evident in several Portuguese districts, such as Lisbon (+11.4%), Porto (+11.3%), and Faro (+23.9%). This has led to the emergence of a pressing challenge for law enforcement agencies and the academic community (as a scientific partner), namely the prevention of firearms. The SafeGuard system, depicted in Figure 1, operates by analyzing video frames from CCTV footage in real time. When potential threats or firearm-related activities are detected, the system generates alerts, which are promptly displayed on the web interface. Police officers using the application can then rapidly respond to these alerts for swift intervention.

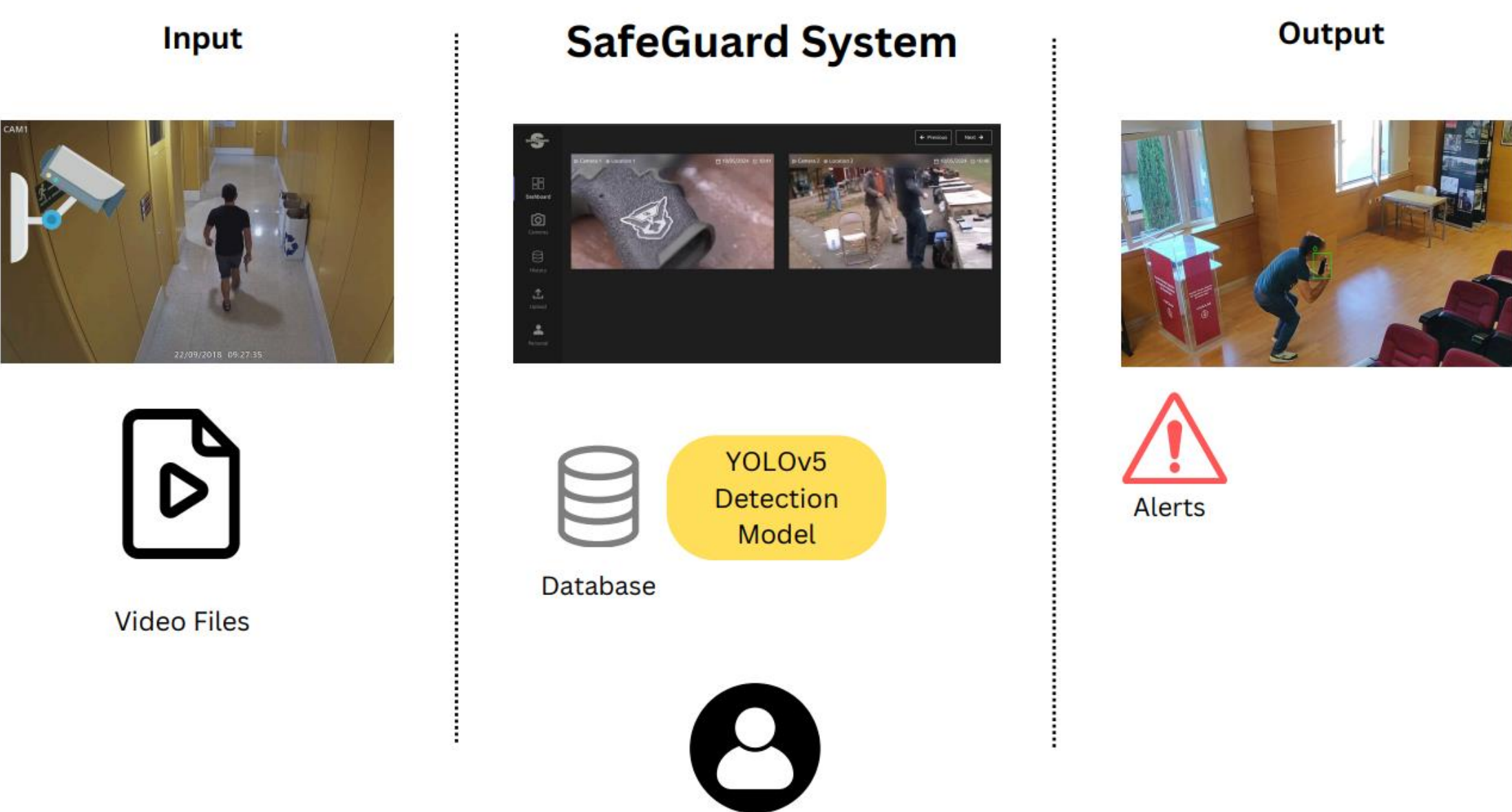


Fig. 1. SafeGuard Application Overview

Objectives

The main objectives can be described as:

- Conduct a thorough analysis of existing research, algorithms, architectures, and datasets in weapon detection;
- Develop a scalable system, SafeGuard, that handles multiple cameras simultaneously, with real-time alerting mechanisms for weapon detection;
- Build a comprehensive and diverse dataset with images and video footage of various weapon types in different environments and conditions.

Methodology

To implement a system capable of detecting weapons, we established the following methodology:

- State of the art review;
- Propose a high-level architecture;
- Establish the architecture with the associated technologies (to resolve the constraints associated with these applications, such as data volume, real-time constraints, and others);
- Propose a detection model using YOLOv5. For this a combined dataset that uses available datasets with some adjustments was developed;
- Application implementation from scratch, and model performing tests.

This analysis led to the development of the architecture, Figure 2, which is detailed below.

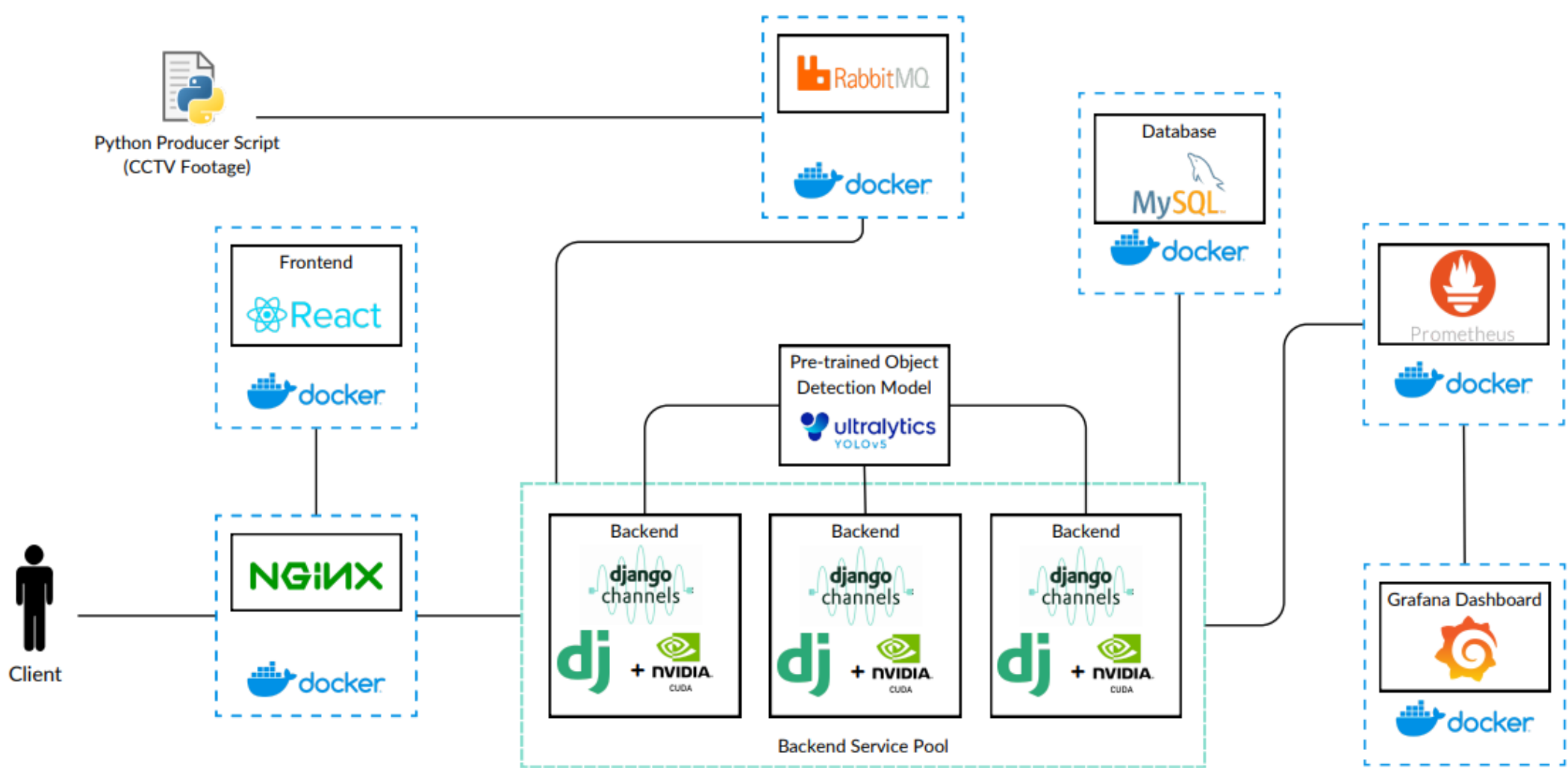


Fig. 2. SafeGuard Implementation Architecture

The initial step was to build a comprehensive dataset containing several scenarios, utilizing three different online datasets. Data augmentation techniques were employed to expand the dataset from its original 11020 images to a total of 22977 images. The augmentation process was conducted in two phases. The first phase involved applying rotations and blurring to the images.

The second phase introduced variations in lighting conditions. These augmentations increased the diversity of the dataset. Following augmentation, the dataset was split into training and testing sets using an 80/20 ratio, as illustrated in Figure 3.

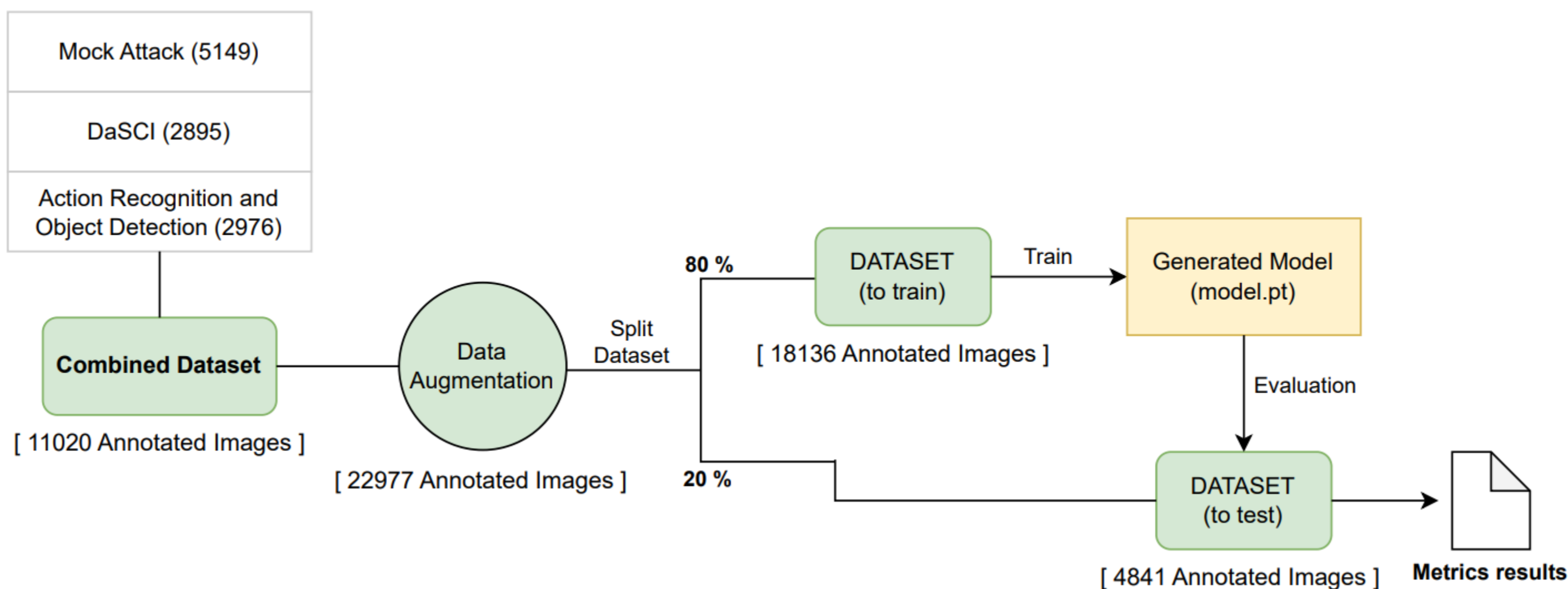


Fig. 3. Model Creation Steps

The architecture components are outlined in the Table 1:

Table 1. System Components and Descriptions

Component	Description
Python Producer Script	Sends simulated footage to RabbitMQ;
RabbitMQ	Ensures smooth and reliable data handling as a message broker;
Django Backend	Utilizes pre-trained YOLOv5 model to process incoming footage; Uses Django Channels for real-time communication and immediate alarm generation based on detected objects; Stores all object detections and user interactions in a MySQL database;
React Frontend	Provides a user-friendly web interface; Allows users to interact with the system, view real-time footage, upload files to analyse;
Load Balancing with Nginx	Manages incoming traffic efficiently; Distributes load across multiple servers; Ensures high availability and scalability of the application;
Performance Monitoring	Prometheus used for real-time performance monitoring; Grafana offers a detailed dashboard for visualizing metrics.

Tests and Results

The YOLOv5 model was trained under various configurations, adjusting parameters such as the number of epochs, batch size, and image size to find the optimal performance. After extensive testing and parameter adjustments, the model optimal performance is represented in Table 2, reaching a maximum accuracy of 0.973.

Table 2. Optimal Configuration Parameters and Results

Epochs	Batch Size	Image Size	Accuracy	Precision	Recall
100	32	640	0.973	0.969	0.944

Figure 4 exhibits an output from one dataset frame, processed using the fine-tuned YOLOv5 model. The model has annotated four objects with bounding boxes, identifying them as potential weapons with varying degrees of confidence: one at 0.61, and two others at 0.65 and 0.75, and a knife at a confidence of 0.55.



Fig. 4. Annotated Model Frame

Conclusions

The system is scalable and can handle multiple cameras at once. This makes it ideal for police officers to use for continuous monitoring. With this system, they can keep an eye on various areas at the same time, helping them prevent crime and respond quickly to incidents. Additionally, this work resulted in the compilation and publication of a comprehensive dataset, which can be used by other developers.

References

- [1] Secret rio-Geral do Sistema de Segurana Interna. (2022). Relatório Anual de Segurana Interna. Retrieved from https://www.ssi.gov.pt/sites/default/files/2023-07/20230331_pm_RASI_2022.pdf
- [2] Piotr Matczak, Andrzej W jtowicz, Adam Dbrowski, and Krzysztof Mczka. (2022). Cost-Effectiveness of CCTV Surveillance Systems: Evidence from a Polish City. Springer, European Journal on Criminal Policy and Research. Retrieved from: <https://link.springer.com/article/10.1007/s10610-022-09527-5>

ACKNOWLEDGEMENTS

This work is funded by FCT (Foundation for Science and Technology) in the context of the project doi.org/10.54499/UIDB/00127/2020.